

실시간 피드백을 통한 절차형 VR 교육 시스템



저자 1 201924407 강원우

저자 2 201824594 조현진

지도교수 이명호

목 차

1. 서론.....	5
1.1. 연구 배경 : VR 교육 시스템의 등장.....	5
1.2. 기존 문제점.....	6
1.2.1. 절차 피드백의 부족.....	6
1.2.2. 실시간 피드백의 부족.....	6
1.2.3. 사용자 질문 응답 구조의 한계.....	6
1.2.4. 문제 해결 중심의 학습 부족.....	7
1.3. 연구 목표.....	7
1.1.1. 절차적 과정에 대한 피드백 강화.....	7
1.1.2. 실시간 피드백 및 사용자 상호작용 강화.....	7
2. 연구 배경.....	7
2.1. VR 교육 시스템의 부상과 실효성.....	8
2.2. 절차 기반 실시간 피드백의 필요성.....	8
2.3. AI Agent 도입의 필요성과 차별성.....	9
2.4. 연구 기획.....	10
2.4.1. 화재 대피 교육 시스템 선정 및 시나리오 기준.....	10
2.4.2. 전체 기능 설계 표.....	10
2.4.3. 개발 환경.....	11
2.4.3.1. 사용 언어.....	11
2.4.3.2. 사용 프로그램.....	11
2.4.3.3. 사용 장비.....	11

2.5.	배경 지식.....	12
2.5.1.	용어.....	12
2.5.2.	Oculus Quest2 컨트롤러 명칭 정리.....	12
3.	연구 내용.....	13
3.1.	시스템 구성도.....	13
3.1.1.	사용자 (VR HMD).....	13
3.1.2.	교육 시스템 (Unity).....	13
3.1.3.	음성 처리 (Google TTS/STT).....	14
3.1.4.	AI 처리 (Gemini API).....	14
3.2.	씬 구성도 및 시나리오.....	14
3.2.1.	Scene #1 시작 화면.....	15
3.2.2.	Scene #2-1 대피 훈련(맵 A) 구성도 및 시나리오.....	15
3.2.3.	Scene #2-2 대피 훈련 (맵 B).....	16
3.2.4.	Scene #3 소화기 훈련.....	18
3.3.	UI 구성.....	19
3.3.1.	플레이 UI.....	19
3.3.2.	결과 UI.....	19
3.3.3.	AI Agent UI.....	20
3.4.	AI Agent 구현.....	21
3.4.1.	User To AI.....	21
3.4.2.	AI 처리.....	22
3.4.3.	AI To User.....	23
3.4.4.	오브젝트 찾기.....	24
3.5.	절차 피드백 구현.....	25

3.5.1.	실시간 피드백.....	26
3.5.2.	절차 보장 (시퀀스 락 시스템).....	26
3.5.3.	End UI와 연동.....	26
3.6.	연습모드와 평가모드.....	26
3.6.1.	모드 구분.....	26
3.6.2.	기능 차이.....	27
4.	연구 결과 분석 및 평가.....	28
4.1.	전체 코드 및 기능 정리.....	28
4.2.	AI Agent 응답 성능 평가.....	28
4.2.1.	분석 그래프.....	29
4.2.2.	분석 결과 표.....	30
5.	결론 및 향후 연구 방향.....	30
5.1.	결론.....	30
5.2.	향후 연구 방향.....	30
구성원별 역할 및 개발 일정		31
개발 일정		31
역할 분담		31
참고 문헌		32

1. 서론

1.1. 연구 배경 : VR 교육 시스템의 등장

교육 시스템은 기술의 발전에 따라 크게 변화해왔습니다. 초기의 텍스트와 영상 중심의 교육은 정보 전달에는 효과적이었으나, 학습자가 실제로 필요한 기술을 체험하고 반복적으로 연습하는 데는 한계가 있었습니다. 이러한 문제를 해결하기 위해 등장한 것이 VR(가상현실) 기술을 활용한 교육 시스템입니다. VR은 사용자가 직접 참여하여 상황을 경험하고 실습할 수 있는 환경을 제공하여 몰입감과 현실감을 극대화할 수 있습니다.

VR 교육은 현실에서 경험하기 어려운 환경을 가상 공간에서 체험할 수 있도록 지원하여 학습자의 이해도를 높이고 실습 효과를 극대화하는 장점이 있습니다. 기존 연구에서는 VR 교육이 다음과 같은 효과를 보였다고 보고되었습니다.

학습 효율 향상: VR을 활용한 학습이 기존의 동영상 및 텍스트 기반 학습보다 학습자 몰입도를 증가시키고, 실습 중심의 학습으로 성취감을 높이는 효과가 있음.

실제 환경과 유사한 경험 제공: 현실에서 재현하기 어려운 방사선 대응 훈련, 공정 실습, 의료 교육 등에서 실질적인 학습 효과를 기대할 수 있음.

반복 학습 가능: VR에서는 학습자가 원하는 만큼 특정 과정을 반복하여 실습할 수 있으며, 이를 통해 숙련도를 향상시킬 수 있음.

특히 방사선 비상 대응에 VR 교육을 적용한 연구에서는 교육생들의 몰입도 증가와 실질적인 숙련도 향상이 보고되었습니다[1]. 이를 시각적으로 표현한 연구 결과를 아래 그래프에서 확인할 수 있습니다.

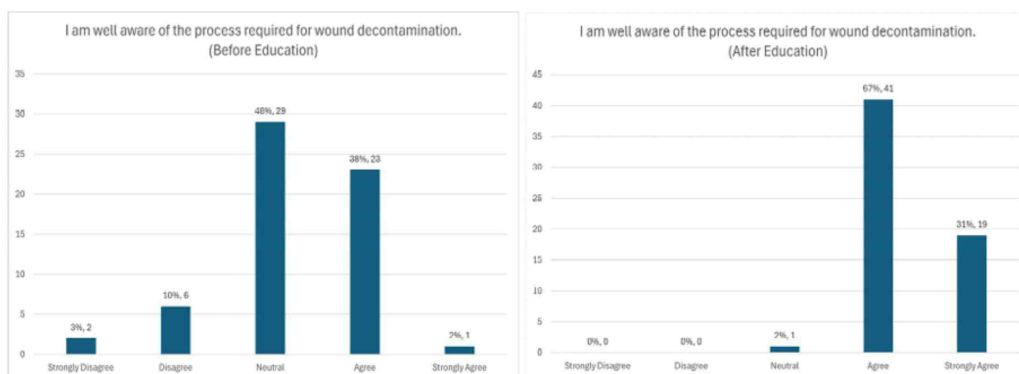


Figure 상처제염 과정에 대한 VR교육 전·후비교[1]

해당 그래프는 "상처 제염 절차에 대해 잘 알고 있는가?"라는 항목에 대한 설문 응답을

교육 전후로 비교한 것으로, VR 교육의 학습 효과를 정량적으로 보여줍니다.

왼쪽 그래프(교육 전)에서는 전체 응답자 중 48%가 '보통이다(Neutral)', 28%가 '그렇다(Agree)'고 응답하였으며, '매우 그렇다(Strongly Agree)'는 단 2%에 불과했습니다. 이는 대부분의 학습자가 훈련 전에는 절차에 대한 확신이 부족했음을 시사합니다.

반면 오른쪽 그래프(교육 후)에서는 '그렇다(Agree)'응답이 67%, '매우 그렇다(Strongly Agree)'가 31%로 나타났습니다. 특히 '보통' 이하 응답(Neutral, Disagree, Strongly Disagree)이 거의 사라진 것을 통해, 훈련을 통해 학습자들의 확신과 절차 이해도가 크게 향상되었음을 알 수 있습니다.

이러한 결과는 VR 기반 교육이 학습자에게 추상적인 정보 전달을 넘어서, 절차적 개념의 명확한 체득과 실질적 자신감을 부여하는 데 효과적임을 실증적으로 뒷받침합니다.

1.2. 기존 문제점

기존 VR 교육 시스템은 많은 장점을 제공하지만, 다음과 같은 한계로 인해 효과가 제한되고 있습니다.

1.2.1. 절차 피드백의 부족

기존의 교육 시스템은 학습자가 특정 절차를 정확히 수행했는지 추적하는 기능이 부족합니다. 절차가 중요한 상황에서는 각 단계의 정확한 수행이 필수적이지만, 일반적인 VR 교육 시스템은 단순히 동작의 유무만을 평가하여 실수의 원인을 파악하기 어렵습니다. 예를 들어, 화재 대피 시 사용자가 문 손잡이를 만지기 전 온도를 확인하지 않는 실수를 했을 때, 이를 즉시 감지하고 피드백하는 시스템이 부족하여 실수가 반복될 가능성이 높습니다.

1.2.2. 실시간 피드백의 부족

일부 기존 VR 훈련 시스템은 기술적으로 실시간 피드백이 가능함에도 불구하고, 실제 구현에서는 주로 단순 시나리오 안내 또는 결과 중심의 정오 판단에 초점을 맞추는 경우가 많아, 학습자가 훈련 도중 발생하는 절차 오류나 행동 실수에 대해 즉시 피드백을 받지 못하는 한계가 존재했습니다.

VR은 본래 사용자 행동을 추적하고 즉시 반응을 제공하기에 적합한 환경이지만, 많은 상용 또는 연구 기반 시스템에서는 피드백 기능이 축약되거나 수동적으로 처리되는 경우가 적지 않았습니다.

1.2.3. 사용자 질문 응답 구조의 한계

훈련 중에는 사용자에게 즉각적인 의문이나 혼란이 발생할 수 있지만, 기존 교육 시스템은 이러한 질문에 대해 실시간으로 응답하거나 유연하게 대화할 수 있는 구조를 갖추

지 못한 경우가 많습니다. 이는 단순한 '응답 속도'의 문제가 아니라, 시스템이 학습자의 질문을 정확히 이해하고 의도를 파악해 적절히 응답하는 데 한계가 있었기 때문입니다.

예를 들어 CBT나 기존 VR 훈련 시스템에서는 사용자의 질문이 예상된 선택지 외의 자유로운 문장일 경우 대응하지 못하거나, 매뉴얼적 가이드만 반복하는 방식으로 응답의 자연스러운 흐름이나 대화의 맥락을 유지하기 어려웠습니다.

이러한 구조적 한계는 학습자가 질문을 포기하거나 잘못된 절차로 훈련을 계속하게 되는 원인이 될 수 있으며,

결과적으로 상황에 대한 이해도와 몰입도를 저하시킬 수 있습니다.

1.2.4. 문제 해결 중심의 학습 부족

기존의 VR 교육 시스템은 주로 단순한 지식 전달에 집중되어 있어 복잡한 문제 상황에서 사용자가 스스로 문제를 해결하는 경험을 제공하지 못하는 경우가 많습니다. 예를 들어, 화재 상황에서 단순히 비상구를 찾아가는 것만이 아니라 연기와 열기를 피하고, 적절한 도구를 활용하여 탈출 경로를 확보하는 등의 실제 상황과 유사한 복잡한 문제 해결 능력을 기르는 데 한계가 있습니다. 이는 단순히 절차를 암기하는 것보다 실질적인 위기 대처 능력을 기르는 데 있어 중요한 요소입니다.

1.3. 연구 목표

1.1.1. 절차적 과정에 대한 피드백 강화

각 절차의 정확한 수행을 실시간으로 추적하고 피드백하는 시스템을 개발하여 사용자가 올바른 절차를 반복 학습할 수 있도록 지원합니다. 이를 통해 사용자는 위기 상황에서 필요한 기술과 절차를 체계적으로 학습하고 실수의 빈도를 줄일 수 있습니다.

1.1.2. 실시간 피드백 및 사용자 상호작용 강화

사용자의 실시간 반응에 따라 즉각적인 피드백을 제공하고, 학습자가 궁금한 점이 있을 때 AI 에이전트를 통한 실시간 응답 시스템을 도입하여 학습의 질을 향상시킵니다. 이를 통해 학습자는 단순한 절차 수행을 넘어, 복잡한 상황에서도 스스로 문제를 해결할 수 있는 능력을 기르게 됩니다.

2. 연구 배경

2.1. VR 교육 시스템의 부상과 실효성

기술의 발전은 교육 방식의 구조적 전환을 촉진해 왔습니다. 초창기의 텍스트 기반 교육은 정보 전달에는 효과적이었지만, 학습자가 실제 기술이나 상황을 직접 체험하고 숙련도를 높이는 데에는 명백한 한계가 있었습니다. 이어서 등장한 영상 기반 학습은 시청

각 자극을 통해 몰입을 강화했지만, 여전히 행동 기반 학습이나 실습형 피드백을 제공하기에는 부족했습니다.

이에 따라 교육계는 학습자 중심의 몰입형 훈련 방식으로 VR(Virtual Reality)기술을 주목하게 되었습니다. VR은 실제 환경을 가상으로 구성하여 학습자가 직접 상황을 체험하고, 반복 수행을 통해 절차와 기술을 습득할 수 있도록 합니다.

기존 연구들은 VR 기반 교육이 실제 학습 성과 향상에 긍정적인 영향을 미친다고 보고합니다. 예를 들어 Shin & Cho(2019)[3]는 VR 훈련 참여자가 텍스트 기반 학습자보다 72% 더 높은 절차 완수율을 보였다고 밝혔고, Park et al.(2020)[4]는 "기억 유지율, 반응 속도, 위기 인식도" 측면에서 기존 방식 대비 VR의 학습 효과가 뛰어나다고 평가했습니다.

그러나 현재까지 보급된 다수의 VR 교육 시스템은 다음과 같은 제한점을 가지고 있습니다:

- 대부분의 시스템은 단순 시나리오 안내나 동작 유도에 머무르며, 학습자의 절차적 판단 오류를 감지하거나 피드백하지 못합니다.
- 시스템의 목적이 "시뮬레이션 체험"에 한정되어, 사용자가 잘못된 순서로 행동하더라도 이를 교정하거나 기록하지 않는 경우가 많습니다.

2.2. 절차 기반 실시간 피드백의 필요성

교육 훈련에서 "무엇을 했는가"만큼 중요한 것은 "어떻게, 어떤 순서로 했는가"입니다. 특히 의료 기술, 군사 전술, 산업 안전, 항공 정비 등 수많은 분야에서 정해진 절차의 정확한 수행이 안전성과 결과에 직접적인 영향을 미칩니다.

절차 학습은 단순 지식 암기와는 달리 순서화된 행동 수행 능력, 즉 "정해진 순서대로 상황에 맞는 행동을 할 수 있는가"를 평가하고 교정할 수 있는 기능이 필요합니다. 하지만 현재 대부분의 교육 시스템에서는 다음과 같은 한계가 존재합니다:

- **오류 감지 부재** : 기존의 CBT 시스템이나 E-learning 플랫폼은 학습자의 정답/오답만을 평가할 뿐, 어떤 단계에서 오류가 발생했는지를 파악하거나 피드백하는 기능을 제공하지 않습니다.
- **동시적 순서 추적 기능 미비** : 예를 들어 의료 분야의 VR 트레이닝 시스템도 사용자 행동을 순차적으로 체크하는 기능보다는 "단계 수행 여부"만을 확인하는 데 그치는 경우가 많습니다.
- **실시간 교정 불가** : 대부분의 시스템은 잘못된 절차 수행을 단지 기록으로 남기거나, 훈련 종료 후에야 평가합니다. 이는 즉각적인 교정 기회를 놓치는 결과를 초래합니다.

실제로 Choi et al.(2021)[5]의 연구에 따르면, 실시간 절차 피드백이 없는 VR 훈련 그

률은 훈련 중 오류 인식률이 낮고, 반복 시에도 같은 실수를 반복하는 경향을 보였습니다. 이는 시스템이 피드백을 제공하지 않으면, 학습자는 자신의 잘못을 인지조차 하지 못하는 경우가 많다는 점을 보여줍니다.

이에 따라 본 연구에서는 다음과 같은 두 가지 문제를 해결하기 위한 시스템을 설계했습니다:

- 학습자의 행동 절차를 실시간으로 추적하고, 순서 오류 발생 시 즉시 피드백 제공
- 시나리오 종료 후에도 전체 절차 수행 이력을 시각적으로 제공하여 학습자가 자기 행동을 점검할 수 있도록 지원

이는 단순 시뮬레이션을 넘어 "행동의 질과 순서를 평가하고 교정하는 절차 중심 학습 구조"를 지향한다는 점에서 기존 시스템과 차별화됩니다.

2.3. AI Agent 도입의 필요성과 차별성

전통적인 교육 훈련 환경에서는 사용자가 훈련 중 궁금한 점이나 이해하지 못한 절차에 대해 즉시 해결할 수 없는 문제가 존재합니다. 특히 시나리오 기반 훈련이나 CBT(Computer-Based Training) 환경에서는 사용자가 훈련 도중 멈추거나 강사의 개입 없이 혼자서 질문을 해결하는 방식이 제공되지 않습니다.

일부 훈련에서는 담당자가 실시간으로 주변을 모니터링하며 피드백을 제공할 수 있으나, 이 방식은 담당자의 상시 배치가 필요하다는 공간적·시간적 제약이 있으며, 다수의 훈련자가 동시에 참여할 경우 모든 질문에 즉각적으로 대응하는 데 한계가 존재합니다.

실제 사례로, 본 논문의 작성자는 예비군 훈련 중 VR 사격 시뮬레이터 훈련을 경험하던 도중 조작 방법에 대한 즉각적인 질문이 있었으나 담당 교관이 근처에 없어 훈련을 멈춘 채 수 분간 대기해야 했습니다. 이 경험은 훈련 도중 실시간 상호작용 시스템의 필요성을 더욱 절감하게 된 계기가 되었습니다.

실제로 Lee & Choi(2020)[6]의 연구에서는 "실습 기반 의료 시뮬레이션에서 강사의 물리적 개입이 지연될 경우 학습자의 혼란이 증가하며, 일부 학습자는 훈련 흐름을 놓치고 소극적인 태도로 전환된다"고 보고되었습니다. 이는 실시간 질문 대응의 부재가 학습 몰입도와 수행 정확도에 부정적인 영향을 줄 수 있음을 의미합니다.

또한 Kwon et al.(2019)[7]은 VR 기반 공정훈련 프로그램을 분석하면서, "사용자들이 오브젝트를 찾지 못하거나 기능의 의미를 이해하지 못해 헤매는 경우가 많았으며, 이 문제는 반복 학습이나 훈련 담당자의 지속적인 개입 없이는 해결되지 않았다"고 지적했습니다.

이러한 한계를 극복하기 위해 본 연구에서는 AI Agent 기반의 실시간 상호작용 시스템을 도입하였습니다.

2.4. 연구 기획

2.4.1. 화재 대피 교육 시스템 선정 및 시나리오 기준

본 과제에서는 절차 피드백, AI 상호작용, 실시간 피드백기능을 효과적으로 구현하고 테스트하기 위해 '화재 대피 훈련 시나리오'를 적용 대상으로 선정하였습니다.

이 시나리오는 다음과 같은 이유로 제안된 기능의 적합성과 효과를 실증하기에 매우 적합하다 판단했습니다.

- **정해진 절차의 정확한 수행이 필수적인 시나리오** : 화재 대피 상황에서는 정해진 절차가 존재하며, 이 절차는 단순 나열이 아니라 정해진 순서로 정확하게 이행되어야 생존율을 높일 수 있습니다. 따라서 절차 피드백 시스템을 평가하기에 이상적인 구조를 갖고 있습니다.
- **행동 오류 발생 시, 실제 피해로 직결되는 구조** : 예를 들어, 문 손잡이의 온도 확인 없이 열거나, 천을 물에 적시지 않은 채 호흡 보호를 시도하는 등 절차 오류는 곧 사고 위험을 의미하므로, 잘못된 행동에 대한 실시간 피드백의 필요성이 매우 뚜렷하게 드러납니다. 이러한 특성은 실시간 피드백 시스템의 효과성을 확인할 수 있는 구체적 조건을 제공합니다.
- **훈련 중 즉각적인 설명·안내의 필요성** : 화재 대피는 낯선 공간, 불확실한 상황, 제한된 시야에서 이루어지기 때문에, 훈련 중에는 사용자가 자주 경로를 잃거나 행동의 이유를 혼동하는 경우가 발생합니다. 이는 음성 기반 AI Agent의 실시간 설명 기능의 유용성을 평가할 수 있는 환경을 자연스럽게 제공합니다.
- **다수의 실제 가이드라인이 존재** : 본 시나리오의 구성과 기준은 다음의 신뢰할 수 있는 공식 자료를 기반으로 설정하였습니다:
 - 국가법령정보센터의 『화재 및 피난 시뮬레이션의 시나리오 작성 기준』[8]
 - 경기도소방재난본부 『원테이크 119초 안전교육 - 화재 대피편』 영상 자료[9]

2.4.2. 전체 기능 설계 표

프로젝트 초기에는 단순한 모듈 기반 설계를 계획했으나, 중간 보고서 이후 졸업과제로서 주제의 명확성과 학술적 의의가 부족하다는 판단에 따라 수정이 이루어졌습니다. 모듈 중심접근법에서 절차 피드백과 실시간 상호작용에 초점을 맞춘 시스템으로 전환했습니다.

문제 상황	기능명	기능 설명
사용자가 잘못된 순서로 행동해도 시스템이 감지하지 않음	절차 기반 시퀀스 트래킹	각 훈련 단계를 정해진 순서로만 수행할 수 있도록 제한하며, 오류 발생 시 기록

훈련 중 실수해도 즉시 피드백이 제공되지 않음	실시간 피드백 시스템	잘못된 행동(예: 문 온도 확인 누락, 포복 미실시 등) 발생 시 UI·햅틱·음성 피드백 제공
훈련이 끝난 뒤 어떤 절차에서 실수했는지 확인 불가	결과 기반 절차 피드백 (End UI 연동)	최종 훈련 결과 화면에서 실수한 절차를 시각적으로 표시하고 구체적인 오차 설명 제공
사용자가 특정 오브젝트를 찾지 못하거나 진행 상황에 막힘	오브젝트 탐색 가이드	사용자가 질문 시 해당 오브젝트 방향 및 위치를 시각적으로 안내 (화살표 + 아웃라인)
훈련 중 궁금한 점에 즉시 대응 불가능	AI 음성 어시스턴트	음성 인식 → 텍스트 변환 → Gemini 응답 생성 → 음성 안내로 연결되는 AI 상호작용 시스템
동일한 훈련 환경 반복 시 학습 효과 저하	맵 랜덤화 시스템	대피 훈련에서 매번 다른 맵이 로딩되도록 하여 외우기 학습 방지
훈련 모드에 따라 피드백 방식이 달라야 함	연습 모드 / 평가 모드 분리	연습 모드에서는 가이드, 피드백 활성화 / 평가 모드에서는 UI 및 피드백 최소화

Table 1 기능 설계 표

2.4.3. 개발 환경

2.4.3.1. 사용 언어

본 프로젝트에서 사용된 주요 프로그래밍 언어는 다음과 같습니다.

- C# - Unity 기반 개발을 위한 주요 언어

2.4.3.2. 사용 프로그램

- **Unity(6000.0.32f1)** : VR 환경 구축과 상호작용 구현을 위한 핵심 플랫폼
- **Visual Studio** : C# 스크립트 작성 및 디버깅
- **Blender** : 3D 모델링 및 에셋 제작
- **Photoshop** : UI 요소 및 텍스처 디자인
- **GitHub** : 코드 버전 관리 및 협업
- **Google TTS/STT** : AI Agent
- **Google Gemini** : AI Agent API
- **Unreal Engine5** : 기존 맵 및 구현 기능 импорт

2.4.3.3. 사용 장비

- **Oculus Quest 2** : VR 테스트와 사용자 인터랙션 검증을 위한 HMD 기기

2.5. 배경 지식

2.5.1. 용어

- **HMD (Head-Mounted Display)** : 사용자가 착용하는 VR 장비로, 가상 환경을 시각적으로 제공
- **XR Interaction Toolkit** : Unity에서 제공하는 기본 VR 상호작용 도구 모음
- **REST API** : 웹 서비스 간의 데이터 통신을 위한 프로토콜
- **AI Agent** : 실시간 사용자 질문에 응답하는 인공지능 모듈
- **절차 피드백** : 사용자의 행동을 절차에 맞게 평가하고 피드백을 제공하는 시스템
- **애셋(Asset)** : 3D 모델, 텍스처, 사운드, 스크립트와 같은 콘텐츠 요소
- **스크립트(Script)** : Unity에서 동작을 제어하는 C# 코드로, 사용자 입력, 물리 효과, AI 상호작용 등을 정의하는 데 사용
- **TTS(Text-to-Speech)** : 텍스트를 음성으로 변환하는 기술로, 본 프로젝트에서는 Google TTS를 사용하여 AI Agent의 응답을 음성으로 출력
- **STT(Speech-to-Text)** : 음성을 텍스트로 변환하는 기술로, 사용자의 음성 명령을 AI가 이해할 수 있도록 변환하는 역할
- **Gemini API** : 사용자의 질문을 처리하고 답변을 생성하는 AI 모델로, 실시간 대화형 AI 시스템의 핵심 컴포넌트
- **리얼리티캡처(RealityCapture)** : 실제 환경의 복잡한 움직임과 상호작용을 보다 현실감 있게 재현하는 기술로, 물리 엔진과 3D 모델의 상호작용을 강화하는 데 사용

2.5.2. Oculus Quest2 컨트롤러 명칭 정리

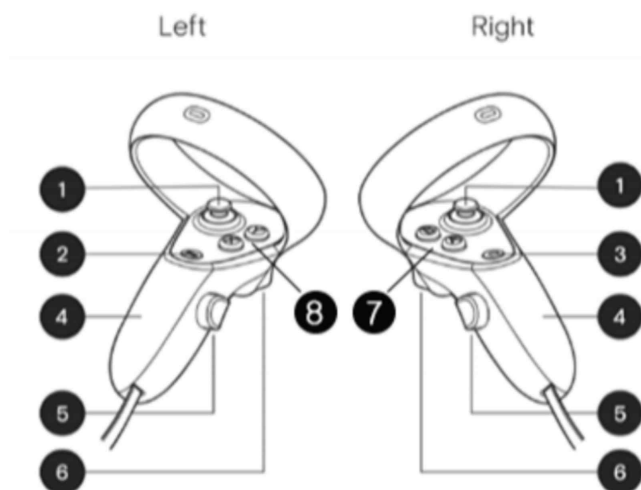


Figure 2 Oculus Quest2 Controller

번호	명칭
1	Thumbstick(엄지스틱)
2	Menu(메뉴 버튼)
3	Oculus(오쿨러스 버튼)
4	Battery cover(배터리 커버)
5	Grip(그립 버튼)

Table 2 Oculus Quest2 컨트롤러 명칭

3. 연구 내용

3.1. 시스템 구성도

본 시스템은 Unity 엔진을 중심으로 사용자와 외부 서비스 간의 실시간 상호작용을 지원하는 VR 교육 플랫폼으로 설계되었습니다. 시스템은 사용자 인터페이스, Unity 내부 모듈, 외부 서비스로 구성되며, 각 모듈 간의 데이터 흐름은 다음과 같습니다.

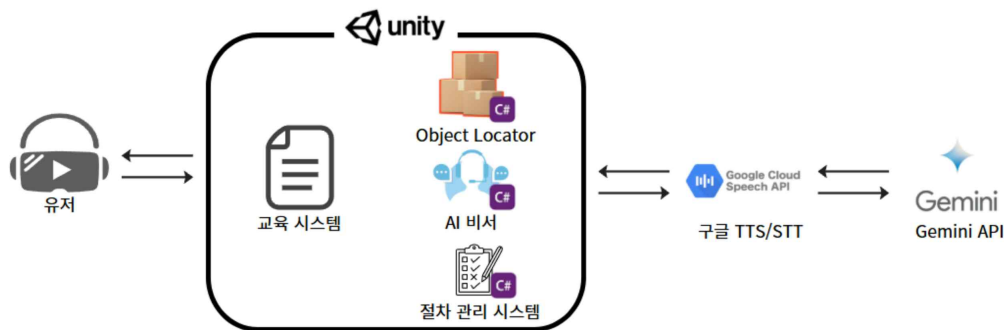


Figure 3 System Design

3.1.1. 사용자 (VR HMD)

사용자는 VR HMD(Head-Mounted Display) 장비를 착용하여 가상 환경에서 대피 훈련을 수행합니다. VR 컨트롤러를 통해 물리적 상호작용이 가능하며, 음성 명령을 통해 AI 비서와 소통할 수 있습니다. 음성 입력은 마이크를 통해 수집되고, 출력은 스피커 및 UI를 통해 사용자에게 전달됩니다.

3.1.2. 교육 시스템 (Unity)

Unity 엔진은 시스템의 핵심 플랫폼으로, VR 환경을 구현하고 내부 모듈 및 외부 서비스를 통합합니다. 주요 하위 모듈은 다음과 같습니다:

- **Object Locator** : 사용자가 특정 오브젝트(예: 소화기, 출입문)를 찾기 어려울 때 위치를 안내하고, 방향을 표시하여 학습을 보조합니다. C# 스크립트로 구현되었으며, ObjectLocator.cs를 통해 동작합니다.
- **AI 비서** : 사용자의 음성 명령을 인식하여 즉각적인 응답을 제공합니다. 음성 데이

터는 Google STT를 통해 텍스트로 변환되고, Gemini API를 통해 질문에 대한 답변을 생성합니다. C#으로 작성된 GeminiManager.cs와 GoogleSpeechToText.cs를 활용합니다.

- **절차 관리 시스템** : 학습자가 절차를 올바르게 수행하는지 실시간으로 모니터링하고 피드백을 제공합니다. 예를 들어, 문 손잡이를 만지기 전 온도 확인, 천을 물에 적시는 등의 절차를 추적하며, SequenceManagerEx.cs를 통해 구현됩니다.

3.1.3. 음성 처리 (Google TTS/STT)

- **Google STT (Speech-to-Text)** : 사용자의 음성 입력을 텍스트로 변환하여 AI 비서가 이해할 수 있는 형태로 전달합니다. 10초간 8kHz로 녹음된 음성 데이터(WavUtility.cs)를 처리하며, GoogleSpeechToText.cs를 통해 통합됩니다.
- **Google TTS (Text-to-Speech)** : AI 비서의 응답을 다시 음성으로 변환하여 사용자에게 전달합니다. 한국어 음성(ko-KR, 볼륨 2.0)으로 출력되며, GoogleTTSManager.cs를 통해 동작합니다.

3.1.4. AI 처리 (Gemini API)

- **Gemini API** : 사용자가 질문한 내용을 실시간으로 처리하고, 적절한 답변을 생성하여 시스템에 반환합니다. Google의 Gemini AI 모델을 활용하며, GeminiManager.cs를 통해 Unity와 통합됩니다. 위치 질문(예: "문이 어디야?")은 ObjectLocator.cs와 연동하여 오브젝트 하이라이트 및 방향 안내를 제공합니다.

3.2. 씬 구성도 및 시나리오

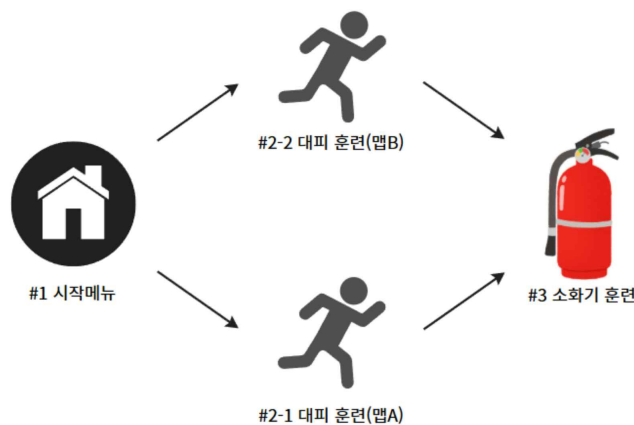


Figure 4 Scene composition diagram

다이아그램은 VR 훈련 시스템의 씬(Scene) 간 전환과 흐름을 보여주는 흐름도로 구성되어 있습니다. 총 4개의 주요 씬이 포함되어 있으며, 각 씬은 특정 훈련 단계를 담당합니다. 씬 간 전환은 화살표로 표시되어 있으며, 사용자가 특정 조건(예: 훈련 완료)을 만족하면 다음 씬으로 이동하거나 이전 씬으로 돌아갈 수 있습니다.

3.2.1. Scene #1 시작 화면

사용자가 학습 모드와 평가 모드를 선택할 수 있는 시작 메뉴입니다. 이 화면에서 대피 훈련 (맵 A, 맵 B) 또는 소화기 훈련을 선택하여 훈련을 시작할 수 있습니다.



Figure 5 Scene#1 UI

3.2.2. Scene #2-1 대피 훈련(맵 A) 구성도 및 시나리오

초기 대피 경로를 따라 이동하는 기본 훈련 시나리오로, 화재 발생 시 기본적인 대피 절차를 학습하는 씬입니다.

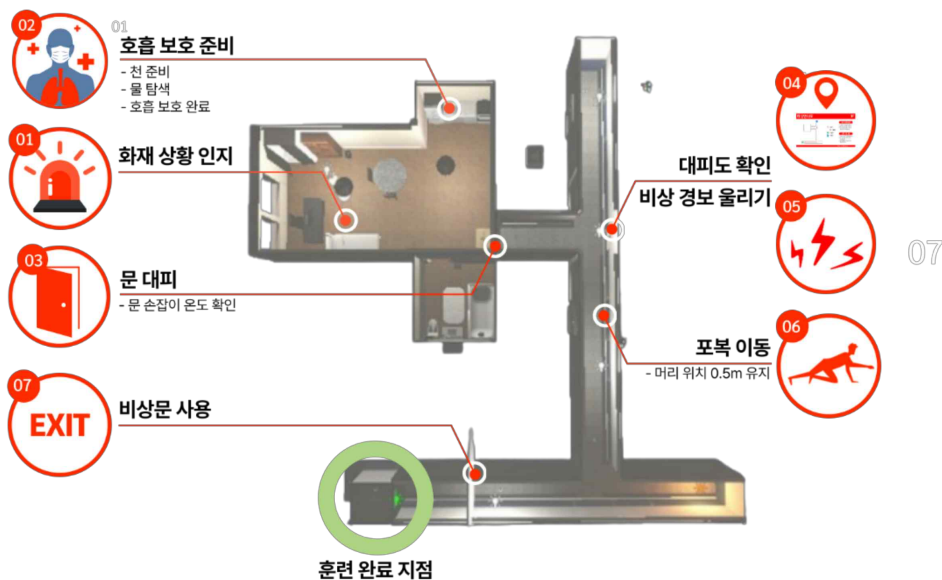


Figure 6 대피 훈련 맵A 구성도

순서	시나리오	상세 설명
1	화재 상황 인지	• 사용자가 VR 환경에서 화재 발생 상황을 인지하고 즉각적인 대응을 시작합니다. • 시각적/음향적 신호(예: 연기, 화재 경보음)를 통해 위기 상황을 감지합니다.
2	호흡 보호 준비	• 물에 적신 천을 이용해 연기 흡입을 방지하는 방법을

		학습합니다. 호흡 보호에 필요한 천 탐색, 물 탐색, 호흡 보호 완료의 절차순으로 진행됩니다.
3	문 대피	- 대피 전 문 손잡이를 3초간 터치하여 온도를 확인하여 반대 편 상황을 확인합니다. - 문을 열고 안전하게 방 외부로 이동합니다.
4	대피도 확인	가상 환경 내 대피도를 확인하여 최적의 탈출 경로를 파악합니다.
5	비상 경보 울리기	비상 경보 버튼을 찾아 눌러 주변 사람들에게 화재를 알립니다.
6	연기 속 포복 이동	- 연기가 가득한 환경에서 포복 자세로 이동하며, 머리 위치를 0.5m 이하로 유지합니다. - 실시간으로 자세를 모니터링하고 피드백을 제공합니다.
7	비상문 사용	비상문을 열고 탈출하며, 안전하게 외부로 이동하는 과정을 완료합니다.
8	시나리오 완료	모든 단계를 성공적으로 완료하면 결과 화면이 표시되며, 경과 시간(TimeManager.cs)과 단계별 성과가 평가됩니다.

Table 3 대피 훈련 맵A 시나리오

3.2.3. Scene #2-2 대피 훈련 (맵 B)

사용자가 동일한 경로에서 반복 훈련을 할 경우, 경로를 암기하여 실제 상황에서의 대응력이 저하될 수 있습니다. 이를 방지하기 위해 새로운 구조의 맵을 추가하여 다양한 시나리오를 제공하며, 사용자가 다양한 환경에서도 올바른 절차를 수행할 수 있도록 설계된 훈련 씬입니다.

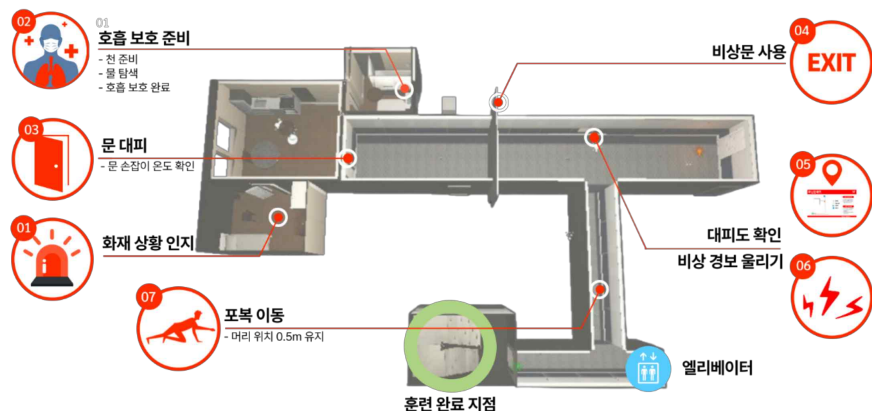


Figure 7 대피 훈련 맵B 구성도

순서	시나리오	상세 설명
1	화재 상황 인지	• 사용자가 VR 환경에서 화재 발생 상황을 인지하고 즉각적인 대응을 시작합니다. • 시각적/음향적 신호(예: 연기, 화재 경보음)를 통해 위기 상황을 감지합니다.
2	호흡 보호 준비	• 물에 적신 천을 이용해 연기 흡입을 방지하는 방법을 학습합니다. • 호흡 보호에 필요한 천 탐색, 물 탐색, 호흡 보호 완료의 절차순으로 진행됩니다.
3	문 대피	• 대피 전 문 손잡이를 3초간 터치하여 온도를 확인하여 반대 편 상황을 확인합니다. • 문을 열고 안전하게 방 외부로 이동합니다.
4	비상문 사용	• 비상문을 열고 탈출하며, 안전하게 외부로 이동하는 과정을 완료합니다.
5	대피도 확인	• 가상 환경 내 대피도를 확인하여 최적의 탈출 경로를 파악합니다.
6	비상 경보 울리기	• 비상 경보 버튼을 찾아 눌러 주변 사람들에게 화재를 알립니다.
7	연기 속 포복 이동	• 연기가 가득한 환경에서 포복 자세로 이동하며, 머리 위치를 0.5m 이하로 유지합니다. • 실시간으로 자세를 모니터링하고 피드백을 제공합니다.
8	엘리베이터 사용	• 사용자가 엘리베이터 사용 시 경고를 하며, 비상 계단 사용을 강조합니다.
9	시나리오 완료	• 모든 단계를 성공적으로 완료하면 결과 화면이 표시되며, 경과 시간(TimeManager.cs)과 단계별 성과가 평가됩니다.

Table 4 대피 훈련 맵B 시나리오

3.2.4. Scene #3 소화기 훈련

사용자가 소화기를 올바르게 사용하는 방법을 학습하는 씬으로, 소화기 들기, 핀 제거, 노즐 조준, 분사 등의 단계를 포함합니다.

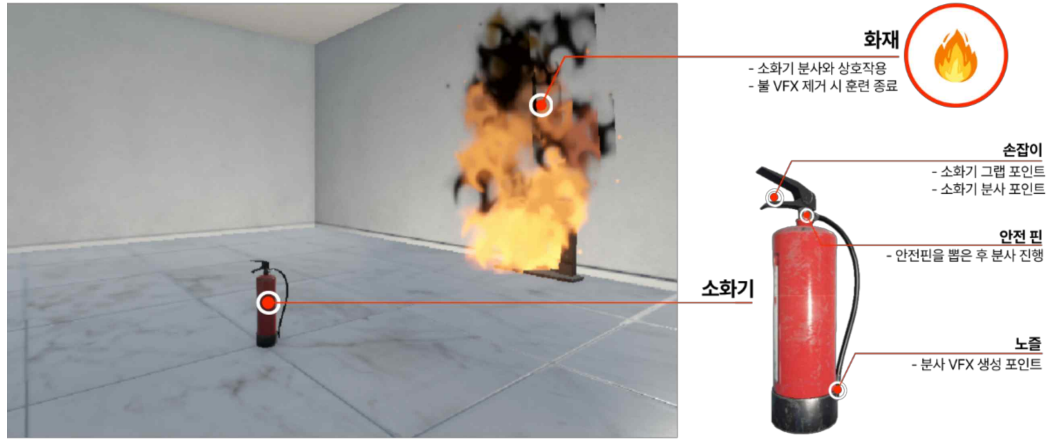


Figure 8 소화기 구성도

순서	시나리오	상세 설명
1	소화기 들기	• 사용자가 VR 환경에서 화재 발생 상황을 인지하고 즉각적인 대응을 시작합니다. • 사용자는 XRGrabInteractable을 통해 소화기를 집습니다.
2	안전핀 제거	• 소화기를 든 사용자가 안전핀 제거 단계로 넘어갑니다. VR 컨트롤러를 사용하여 안전핀을 제거하며, PinController.cs로 해당 동작을 처리합니다.
3	노즐 조준	• 사용자가 소화기의 노즐을 화재 방향으로 조준합니다. 노즐 각도가 30도 이내로 유지되어야 합니다. • 조준이 완료되면 시스템이 자동으로 준비 상태를 확인하고 다음 단계로 진행합니다.
4	소화기 분사	• 사용자가 VR 컨트롤러의 트리거를 눌러 소화기를 분사하며, FireExtinguisherController.cs로 분사 동작을 처리합니다. • 화재가 성공적으로 진압되면 초록색 피드백과 함께 "화재가 진압되었습니다"라는 안내가 제공됩니다.
5	시나리오 완료	• 모든 소화기 사용 단계를 완료하면 시나리오가 종료됩니다. • 결과 화면으로 이동하며, 경과 시간(TimeManager.cs)과 단계별 성공/실패 결과를 표시합니다. • 사용자는 필요 시 이전 단계(Scene #2)로 복귀하거나 훈련을 재시작할 수 있습니다.

Table 5 소화기 훈련 시나리오

3.3. UI 구성

3.3.1. 플레이 UI

플레이 UI는 훈련 프로그램 진행 중 사용자에게 실시간 정보를 제공하며, 효과적인 학습을 지원합니다. 주요 구성 요소는 다음과 같습니다:

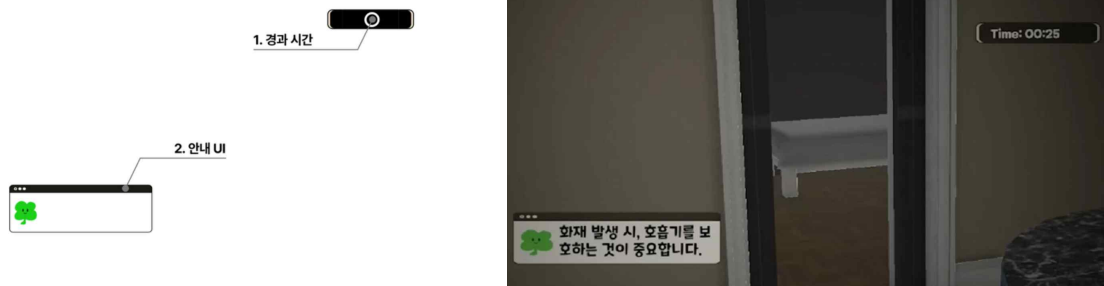


Figure 9 Play UI

- **경과 시간** : 훈련 프로그램 진행 중 경과된 시간을 실시간으로 표시하여 사용자가 훈련 소요 시간을 파악할 수 있도록 합니다.
- **안내 UI** : 현재 진행 중인 시나리오에 대한 설명과 사용자가 다음에 취해야 할 행동 지침을 제공합니다. 또한, 훈련 중 사용자가 잘못된 행동을 할 경우 이를 감지하고 즉각적인 피드백을 표시하여 올바른 절차를 유도합니다.

3.3.2. 결과 UI

결과 UI는 훈련 완료 후 사용자의 성과를 시각적으로 정리하여 제공하며, 사용자가 자신의 수행 능력을 평가하고 개선점을 파악할 수 있도록 돕습니다.

주요 구성 요소는 다음과 같습니다

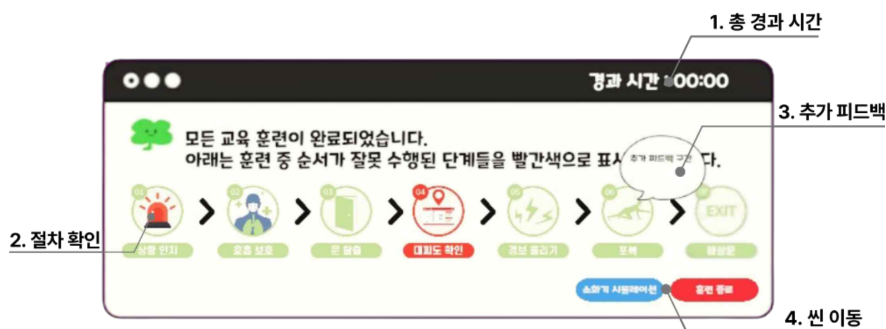


Figure 10 Result UI

- **총 경과 시간** : 훈련 프로그램에 소요된 총 시간을 표시하여 사용자가 전체 훈련 소요 시간을 확인하고 자신의 효율성을 평가할 수 있도록 합니다.

- **절차 확인** : 사용자가 수행한 시나리오의 절차를 시각적으로 나타냅니다. 잘못 수행한 절차가 있을 경우 해당 부분을 빨간색으로 강조하여 사용자가 오류를 쉽게 인지할 수 있도록 합니다.
- **추가 피드백** : 잘못 수행한 시나리오에서 어떤 행동이 잘못되었는지 구체적으로 설명합니다. 사용자가 VR 컨트롤러로 해당 시나리오 이미지를 가리키면(Hover) 추가 피드백이 표시되어 오류의 원인을 상세히 파악할 수 있습니다.
- **씬 이동** : 다른 훈련 씬으로 이동할 수 있는 버튼을 제공하여 사용자가 원하는 시나리오로 쉽게 전환할 수 있도록 지원합니다.

3.3.3. AI Agent UI

AI Agent UI는 사용자와 AI 에이전트 간의 상호작용을 지원하며, 실시간 질의응답을 시각적으로 관리합니다. 주요 구성 요소는 다음과 같습니다:



Figure 11 AI Agent UI

- **AI UI** : AI 에이전트와의 상호작용을 표시하는 UI로, 사용자가 입력한 질문과 AI의 답변을 보여줍니다. 이를 통해 사용자는 자신이 질문한 내용을 확인하고, AI로부터 받은 답변을 파악할 수 있습니다.

3.4. AI Agent 구현

3.4.1. User To AI

AI 에이전트는 사용자의 음성 명령을 수신하여 이를 처리하는 과정으로 시작합니다. 음성 데이터는 VoiceManager.cs와 WavUtility.cs를 통해 수집되며, GoogleSpeechToText.cs를 통해 텍스트로 변환된 후 AI에 전달됩니다. 사용자의 입력이 AI로 전달되는 상세한 절차는 다음과 같이 진행됩니다:

- **음성 수신** : 사용자가 VR 컨트롤러의 지정된 버튼을 누르는 동안 녹음이 진행됩니다. 녹음은 최대 10초 동안 진행되며, 8kHz 샘플레이트로 수집됩니다.

구현 코드

```
[SerializeField] private GameObject recordUI; // 녹음 중일 때 표시되는 UI
public TextMeshProGUI statusText; // 녹음 상태를 표시하는 텍스트
private AudioClip recordedClip; // 녹음된 음성을 저장할 AudioClip
private bool isRecording = false; // 녹음 상태를 추적하기 위한 플래그
private InputDevice rightHandDevice; // 오른손 컨트롤러 장치
private bool previousButtonState = false; // 이전 버튼 상태를 저장하는 플래그

private void Update()
{
    // 오른손 컨트롤러가 유효한지 확인
    if (!rightHandDevice.isValid)
    {
        var rightHandDevices = new List<InputDevice>();
        // 오른손 컨트롤러를 가져옴
        InputDevices.GetDevicesAtXRNode(XRNode.RightHand, rightHandDevices);
        if (rightHandDevices.Count > 0)
            rightHandDevice = rightHandDevices[0];
        return;
    }

    // 기본 버튼 상태를 확인
    if (rightHandDevice.TryGetFeatureValue(CommonUsages.primaryButton, out bool isPrimaryButtonPressed))
    {
        // 버튼이 눌렀고 이전에 눌리지 않은 상태였다면 녹음 시작
        if (isPrimaryButtonPressed && !previousButtonState)
        {
            StartRecording();
        }
        // 버튼 상태를 업데이트하여 다음 프레임에서 비교할 수 있도록 함
        previousButtonState = isPrimaryButtonPressed;
    }

    // 녹음 시작
    private void StartRecording()
    {
        // 이미 녹음 중이면 중복 시작을 방지
        if (isRecording) return;
        // 녹음 상태 플래그 설정
        isRecording = true;
        // UI 업데이트
        statusText.text = "🎤 녹음 중...";
        recordUI.SetActive(true);
        // 마이크로부터 음성을 10초 동안 녹음 (8kHz 샘플링)
        recordedClip = Microphone.Start(null, false, 10, 8000);
    }
}
```

- **데이터 처리** : 수집된 음성 데이터는 WAV 형식으로 변환되며, WavUtility.cs를 통해 바이너리 데이터로 저장됩니다. 이 과정은 음성 클립을 효율적으로 처리하여 후속 변환 단계에 적합한 형태로 준비합니다.

구현 코드

```
// AudioClip에서 샘플 데이터 추출 후 Wav 형식으로 변환
public static byte[] FromAudioClip(AudioClip clip)
{
    float[] samples = new float[clip.samples * clip.channels];
    clip.GetData(samples, 0);
    byte[] wav = ConvertToWav(samples, clip.channels, clip.frequency);
    return wav;
}

// 샘플 데이터를 WAV 형식의 바이트 배열로 변환하는 내부 메서드
private static byte[] ConvertToWav(float[] samples, int channels, int sampleRate)
{
    MemoryStream stream = new MemoryStream();
    BinaryWriter writer = new BinaryWriter(stream);
    writer.Write("RIFF".ToCharArray());
    writer.Write(0); // 파일 크기 플래이스홀더
    writer.Write("data".ToCharArray());
    writer.Write(samples.Length * 2);
    foreach (var sample in samples)
    {
        short intSample = (short)(sample * short.MaxValue);
        writer.Write(intSample);
    }
    stream.Seek(4, SeekOrigin.Begin);
    writer.Write((int)(stream.Length - 8));
    return stream.ToArray();
}
```

- **텍스트로 변환** : 변환된 바이너리 데이터는 GoogleSpeechToText 클래스를 통해 Google STT API로 전송되어 텍스트로 변환됩니다. 이 과정에서 음성 인식 결과는

JSON 형식으로 반환되며, 추출된 텍스트는 AI 처리 단계로 전달됩니다.

구현 코드

```
public static IEnumerator SendAudio(byte[] wavData, System.Action<string> onResult)
{
    string url = $"https://speech.googleapis.com/v1/speech:recognize?key={googleApiKey}";
    string base64Audio = System.Convert.ToBase64String(wavData);
    string jsonBody = $"{{
        \"config\": {{\"sampleRateHertz\": 8000, \"languageCode\": \"ko-KR\"}},
        \"audio\": {{\"content\": \"{base64Audio}\"}}
    }}";
    UnityWebRequest request = new UnityWebRequest(url, "POST");
    byte[] bodyRaw = Encoding.UTF8.GetBytes(jsonBody);
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
    yield return request.SendWebRequest();
    if (request.result == UnityWebRequest.Result.Success)
    {
        string recognizedText = ParseGoogleSTTResponse(request.downloadHandler.text);
        onResult?.Invoke(recognizedText);
    }
}

private static string ParseGoogleSTTResponse(string json)
{
    if (json.Contains("\"transcript\""))
    {
        int index = json.IndexOf("\"transcript\"") + "\"transcript\"".Length;
        int start = json.IndexOf("\"", index) + 1;
        int end = json.IndexOf("\"", start);
        return json.Substring(start, end - start);
    }
    return null;
}
```

3.4.2. AI 처리

이전 단계에서 변환된 텍스트는 GeminiManager.cs를 통해 Gemini AI로 전달되어 처리됩니다. 이를 통해 AI는 사용자의 질문을 분석하고 적절한 답변을 생성할 수 있습니다. 상세한 절차는 다음과 같습니다.

- **텍스트 전송** : 변환된 텍스트는 Gemini API로 전송되며, 사전 정의된 컨텍스트(예: 화재 대피 시나리오)를 기반으로 질문의 의도를 파악합니다.

구현 코드

```
public IEnumerator SendPrompt(string question, System.Action<string> onResponse)
{
    string url = $"https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-pro:generateContent?key={geminiApiKey}";
    StringBuilder sb = new StringBuilder();
    sb.Append("{\"contents\": [{\"role\": \"user\", \"parts\": [{\"text\": \"\"}]}]");
    sb.Append(EscapeJson(question));
    sb.Append("\"}");
    string jsonBody = sb.ToString();
    UnityWebRequest request = new UnityWebRequest(url, "POST");
    byte[] bodyRaw = Encoding.UTF8.GetBytes(jsonBody);
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
    yield return request.SendWebRequest();
    if (request.result == UnityWebRequest.Result.Success)
    {
        string answer = ParseGeminiResponse(request.downloadHandler.text);
        onResponse?.Invoke(answer);
    }
}

private string EscapeJson(string str)
{
    return str.Replace("\"", "\\\"");
}
```

- **답변 생성** : AI는 질문에 따라 응답을 생성하며, 위치 관련 질문은 ObjectLocator.cs와 연동되어 공간 내 오브젝트 정보를 반영합니다. 최대 3회의 재시도 로직을 통해

네트워크 오류를 방지합니다.

- **과정 추가** : 생성된 답변은 JSON 형식으로 반환되며, ParseGeminiResponse메서드를 통해 추출되어 후속 단계로 전달됩니다.

구현 코드

```
private string ParseGeminiResponse(string json)
{
    // 간단 파싱: "text" 필드 추출
    if (json.Contains("\"text\""))
    {
        int index = json.IndexOf("\"text\"") + "\"text\"".Length;
        int start = json.IndexOf("\"", index) + 1;
        int end = json.IndexOf("\"", start);
        string text = json.Substring(start, end - start);
        Debug.Log($"[GeminiManager] AI 답변 수신: {text}");
        return text;
    }
    else
    {
        Debug.LogWarning("[GeminiManager] AI 답변 없음");
        return "답변을 가져올 수 없습니다.";
    }
}
```

3.4.3. AI To User

AI로부터 전달받은 답변 텍스트는 GoogleTTSManager.cs를 통해 음성으로 변환되어 사용자에게 전달됩니다. 이 단계에서는 Google TTS 기술을 활용하여 자연스럽고 명확한 한국어 음성을 생성합니다. 상세한 절차는 다음과 같습니다.

- **텍스트 변환** : 답변 텍스트는 Google TTS API로 전송되며, ko-KR 언어 코드와 NEUTRAL 성별로 설정된 음성으로 변환됩니다.

구현 코드

```
//Google TTS (Text-to-Speech) API를 사용하여 입력된 텍스트를 음성으로 변환하여 재생하는 메서드
public IEnumerator Speak(string text)
{
    string url = $"https://texttospeech.googleapis.com/v1/text:synthesize?key={ttsApiKey}";
    var requestBody = new { input = new { text = text }, voice = new { languageCode = "ko-KR", ssmlGender = "NEUTRAL" }, audioConfig = new { audioEncoding = "LINEAR16" } };
    string jsonBody = JsonUtility.ToJson(requestBody);
    UnityWebRequest request = new UnityWebRequest(url, "POST");
    byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(jsonBody);
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
    yield return request.SendWebRequest();
    if (request.result == UnityWebRequest.Result.Success)
    {
        string base64Audio = ParseTTSResponse(request.downloadHandler.text);
        PlayAudio(base64Audio);
    }
}

//Google TTS API 응답에서 base64 인코딩된 오디오 데이터를 추출하는 메서드
private void PlayAudio(string base64)
{
    byte[] audioBytes = System.Convert.FromBase64String(base64);
    AudioClip audioClip = AudioClip.Create("TTS_Audio", audioBytes.Length / 2, 1, 8000, false);
    audioClip.SetData(System.IO.File.ReadAllBytes("temp.wav"), 0);
    AudioSource audioSource = gameObject.AddComponent();
    audioSource.clip = audioClip;
    audioSource.Play();
}
```

- **텍스트 전송** : 변환된 텍스트는 Gemini API로 전송되며, 사전 정의된 컨텍스트(예: 화재 대피 시나리오)를 기반으로 질문의 의도를 파악합니다.
- **음성 재생** : 변환된 오디오는 VR 헤드셋의 오디오 소스를 통해 2.0 배 볼륨으로 재생되며, 헤드셋 위치를 기준으로 균등한 출력이 보장됩니다.
- **과정 추가** : 재생 후 5초 대기 후 UI가 원래 상태로 복귀하며, 오디오 객체는 자동 삭제됩니다.

3.4.4. 오브젝트 찾기

AI 에이전트는 사용자로부터 받은 질문이 씬 내 오브젝트(예: 소화기, 문, 비상구)를 찾는 요청인지 판단합니다. 이 경우 ObjectLocator.cs와 OutlineTarget.cs를 활용하여 해당 오브젝트를 식별하고 시각적 가이드를 제공합니다. 상세한 절차는 다음과 같습니다:

- **질문 분석** : CheckAndHighlight 메서드를 통해 입력 텍스트에서 위치 관련 키워드 (예: "어디", "위치")와 오브젝트 명을 탐지합니다.

구현 코드

```
// 위치 질문으로 인식할 키워드 (확장 가능)
private readonly string[] locationHints = {
    "어디", "위치", "못 찾겠어", "보이지", "찾을 수 없어", "안 보여" };

public bool IsLocationQuestion(string userInput, out string matchedKeyword)
{
    matchedKeyword = null;
    if (string.IsNullOrEmpty(userInput)) return false;

    foreach (var mapping in keywordMappings)
    {
        string keyword = mapping.keyword;

        if (userInput.Contains(keyword))
        {
            foreach (var hint in locationHints)
            {
                if (userInput.Contains(hint))
                {
                    matchedKeyword = keyword;
                    return true;
                }
            }
        }
    }

    return false;
}
```


- **오브젝트 하이라이트** : 일치하는 오브젝트가 발견되면 OutlineTarget.cs를 통해 3초 동안 윤곽선을 표시하며, 사용자가 방향을 인지할 수 있도록 합니다.

구현 코드

```
private Outline outline;

private void Awake()
{
    outline = GetComponent<Outline>();
    if (outline != null) outline.enabled = false;
}

public void Highlight(float duration = 3f)
{
    if (outline == null) return;
    outline.enabled = true;
    CancelInvoke();
    Invoke(nameof(Disable), duration);
}

private void Disable()
{
    if (outline != null) outline.enabled = false;
}
```

- **방향 안내** : GetRelativeDirectionTo메서드를 통해 플레이어의 시점에서 오브젝트의 상대 방향(예: "앞", "오른쪽")을 계산하여 AI 응답에 포함합니다.

구현 코드

```
public string GetRelativeDirectionTo(string keyword)
{
    if (string.IsNullOrEmpty(keyword) || playerHeadTransform == null)
        return "주변에 위치해 있습니다";

    // 대상 찾기
    var target = keywordMappings.Find(k => k.keyword == keyword)?.target;
    if (target == null) return "주변에 위치해 있습니다";

    Vector3 toTarget = (target.transform.position - playerHeadTransform.position).normalized;
    Vector3 forward = playerHeadTransform.forward;

    // 평면 방향 (수평 판단만)
    toTarget.y = 0;
    forward.y = 0;

    float angle = Vector3.SignedAngle(forward, toTarget, Vector3.up);

    if (angle >= -45f && angle < 45f) return "앞";
    if (angle >= 45f && angle < 135f) return "오른쪽";
    if (angle >= -135f && angle < -45f) return "왼쪽";
    return "뒤";
}
```

3.5. 절차 피드백 구현

절차 피드백 시스템은 사용자가 화재 대피 훈련 중 각 단계의 절차를 정확하게 이행하고 있는지 실시간으로 평가하고 피드백을 제공하는 핵심 모듈입니다. 이 시스템은 학습자가 올바른 순서대로 행동하도록 유도하며, 절차 오류를 자동으로 감지하고 이를 기록하여 최종 평가 결과에 반영합니다. 이를 통해 단순한 반복 학습을 넘어서 절차 기반의 실제 대응 능력을 키우는 데 목적이 있습니다. 이 시스템은 아래와 같은 세 가지 핵심

기능으로 구성됩니다:

3.5.1. 실시간 피드백

훈련 도중 사용자가 잘못된 절차를 수행하면, 즉시 Play UI를 통해 경고 메시지를 시각적으로 전달합니다. 예를 들어, 문 손잡이를 만지기 전에 온도를 확인하지 않으면 화면 상단에 "온도 확인 필요"라는 메시지가 출력됩니다. 이 경고는 음성 안내, 텍스트 안내, UI 강조(빨간 테두리 등)를 통해 제공될 수 있으며, 피드백이 늦거나 누락되는 것을 방지하기 위해 GoogleTTSManager.cs와 연동하여 음성 안내도 출력합니다.

3.5.2. 절차 보장 (시퀀스 락 시스템)

절차 기반 훈련의 정확성을 보장하기 위해, 이전 단계가 완료되지 않으면 다음 단계로 진행할 수 없도록 락 시스템이 구현되어 있습니다. 예를 들어, 천을 물에 적시지 않고 호흡 보호를 시도하면 해당 시퀀스는 실행되지 않으며, UI와 음성을 통해 다시 이전 절차로 돌아가야 함을 안내합니다. 이 기능은 SequenceManager.IsStepCompleted(stepIndex) 메서드로 구현되며, 각 시나리오 스크립트 내에서 조건 분기문을 통해 제어됩니다.

3.5.3. End UI와 연동

훈련 종료 후에는 End UI를 통해 사용자의 전체 절차 수행 내역을 시각적으로 요약하여 보여줍니다. 이때, 사용자가 어떤 절차를 누락했거나 잘못 수행했는지를 구체적으로 표시합니다.

- 각 절차에 대해 성공 여부를 표시 (예: 체크 표시 또는 X 표시)
- 오류가 발생한 절차는 빨간색으로 강조
- 마우스나 컨트롤러로 해당 절차 아이콘을 Hover하면 상세한 피드백이 툴팁으로 출력됨 (예: "문 손잡이 온도 확인 누락")

3.6. 연습모드와 평가모드

연습모드와 평가모드는 각각 학습과 실전 평가라는 목적에 따라 서로 다른 환경을 제공합니다. 사용자는 연습모드를 통해 시나리오를 익히고 기능별 조작법을 학습한 뒤, 평가모드에서 실제 상황과 유사한 환경에서 절차를 수행하게 됩니다.

이를 통해 학습자는 점진적으로 능숙도를 향상시키며 실제 화재 대피 상황에 대한 대응력을 기를 수 있습니다.

3.6.1. 모드 구분

연습모드와 평가모드는 '#Scene1 시작 메뉴'에서 선택할 수 있으며, 선택된 모드는 PlayerPrefs에 저장된 ENUM 값을 통해 다음 씬으로 전달됩니다. 이후 각 씬에서는 이 값을 기반으로 연습모드 또는 평가모드에 맞는 동작을 수행합니다.

구현 코드

```
public enum Mode { Study = 0, Evaluation = 1, NULL }

void SetMode()
{
    int modeValue = PlayerPrefs.GetInt("mode", (int)Mode.NULL);
    currentMode = (Mode)modeValue;

    isPracticeMode = (currentMode == Mode.Study);
}
```

3.6.2. 기능 차이

연습모드에서는 다음과 같은 기능들이 활성화되며, 평가모드에서는 비활성화되어 실제 상황처럼 최소한의 피드백만 제공합니다:

- **다음 행동 가이드** : 각 시나리오 단계에서 사용자가 수행해야 할 다음 행동을 시각적으로 안내합니다. 예를 들어, 소화기 탐색 단계에서는 해당 오브젝트에 아웃라인이 표시되고, 화살표가 방향을 가리키며, 텍스트로 설명이 제공됩니다.
- **실시간 피드백** : 학습자가 잘못된 절차를 수행할 때 즉시 경고 메시지를 표시하는 등의 피드백을 제공합니다. 예시: 문 손잡이를 만지기 전에 온도를 확인하지 않으면 '위험 - 온도 확인 필수'라는 메시지가 출력됩니다.
- **플레이 UI** : 학습 진행 상황, 경과 시간, 현재 절차를 실시간으로 표시하여 학습자가 목표를 명확히 이해할 수 있도록 돕습니다.
- **시퀀스 락 시스템** : 특정 단계가 완료되지 않으면 다음 단계로 넘어가지 않도록 제한합니다. 예시: 천을 물에 적시지 않고 호흡 보호를 시도할 경우, 해당 행동은 무시되며 올바른 절차를 따를 때까지 시스템이 대기 상태로 유지됩니다.
- **AI Agent 활성화** : 연습모드에서는 사용자가 버튼을 눌러 질문하면 AI Agent가 실시간으로 음성 답변을 제공합니다. 예를 들어 "문이 어디야?"와 같은 질문에는 실제 문 방향을 가리키며, 관련 오브젝트를 하이라이트합니다. 반면 평가모드에서는 AI Agent 기능이 제한되거나 완전히 비활성화되어, 사용자가 스스로 판단하고 행동해야 합니다.

4. 연구 결과 분석 및 평가

4.1. 전체 코드 및 기능 정리

기능 분류	스크립트명	주요 기능
훈련 흐름 제어	SceneChanger.cs, SceneManager.cs	훈련 씬 이동, 연습/평가 모드 분기
절차 추적 및 피드백	SequenceManager.cs,	사용자의 시나리오 단계별

	HandComplete.cs	행동 기록, 절차 오류 감지
시각 안내 시스템	OutlineTarget.cs, ObjectLocator.cs	특정 오브젝트에 외곽선 및 화살표 표시로 위치 안내
AI 음성 상호작용	VoiceManager.cs, GoogleSpeechToText.cs, GeminiManager.cs, GoogleTTSManager.cs	사용자 음성 입력 인식, AI 응답 생성 및 음성 출력
UI 구성 및 피드백	EndUI.cs, TimeManager.cs	실시간 시간 표시, 피드백 메시지 출력, 훈련 결과 요약 표시
소화기 기능 제어	FireExtinguisherController.cs, PinController.cs	XR 오브젝트 잡기, 핀 제거, 분사 동작 제어

Table 6 전체 코드 기능 분류

4.2. AI Agent 응답 성능 평가

본 과제에서 구현한 AI Agent는 사용자의 음성 질문을 인식한 후, 이를 텍스트로 변환하여 Gemini API로 전송하고, 생성된 응답을 다시 음성(TTS)으로 출력하는 구조로 설계되어 있다.

시스템의 실시간성과 응답 속도를 평가하기 위해 2개의 질문을 기준으로 각각 10회의 반복 테스트를 수행하였으며, 다음과 같은 시간 요소를 측정하였다:

- **Send Time** : 사용자의 음성이 텍스트로 변환된 후, 서버로 전송되기까지 소요된 시간
- **Response Time** : Gemini API 응답이 도착하고, 이를 음성으로 재생하는 데까지 걸리는 전체 시간
- **Combined Time** : Send Time + Response Time

실험에 사용된 질문 2가지는 다음과 같다.

- **Question 1** : 화재 대피 시 유독가스를 마시면 위험한 이유를 알려줘
- **Question 2** : (호흡 보호 절차에서) 다음으로 해야 하는 행동에 대해 알려줘

4.2.1. 분석 그래프

실험에 사용된 질문 2가지와 이에 해당하는 분석 그래프는 아래와 같다.

- **Question 1** : 화재 대피 시 유독가스를 마시면 위험한 이유를 알려줘

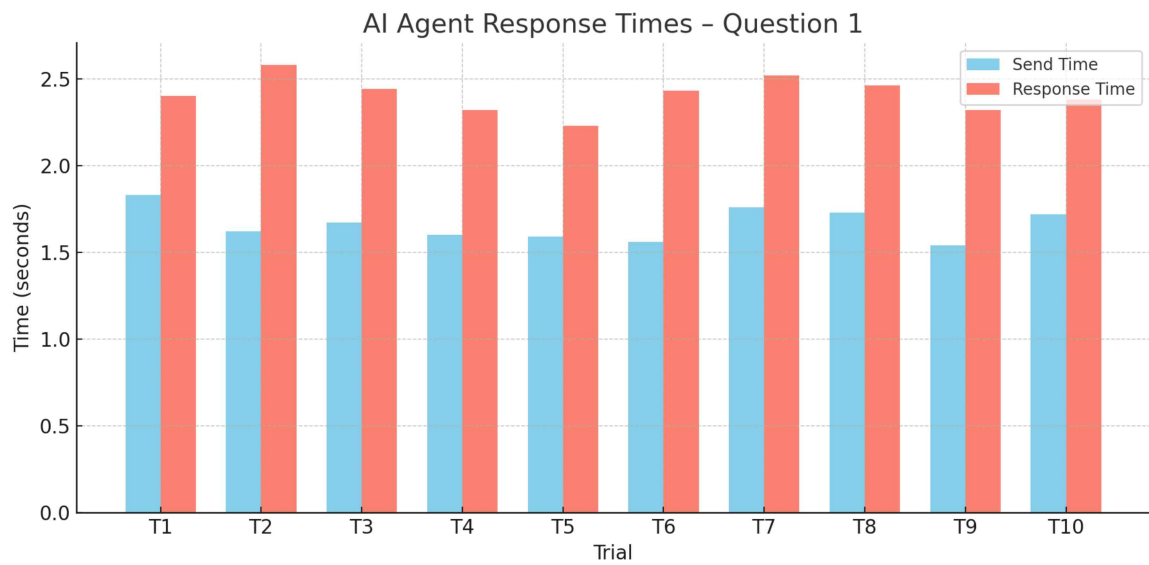


Figure 2 AI Agent Response Times On Question 1

- **Question 2 :** (호흡 보호 절차에서) 다음으로 해야 하는 행동에 대해 알려줘

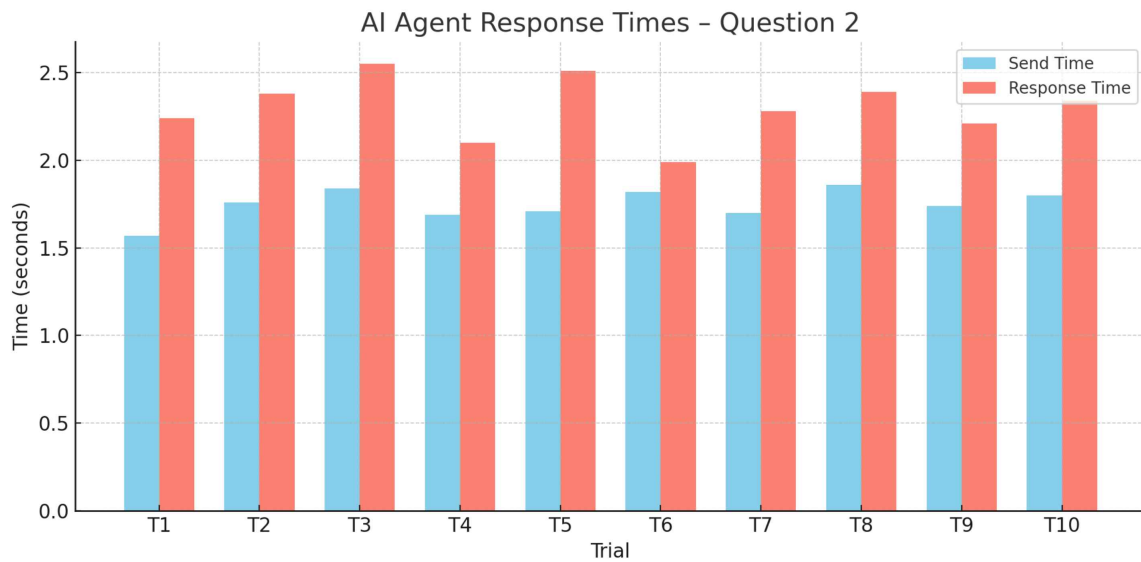


Figure 13 AI Agent Response Times On Question 2

4.2.2. 분석 결과 표

Question	Avg Send Time(s)	Avg Response Time(s)	Combined Avg Time(s)
Q1	1.66	2.41	4.07
Q2	1.75	2.30	4.05
Total Average	1.70	2.36	4.06

Table 7 Analysis Results

5. 결론 및 향후 연구 방향

5.1. 결론

본 연구는 절차 중심의 훈련 교육에서 발생하는 기존 시스템의 한계—즉, 실시간 피드백의 부재, 순서 오류에 대한 감지 부재, 학습자 질문 대응 불가—를 극복하기 위해, 절차 피드백 기능, 실시간 사용자 반응 감지, AI Agent 기반 상호작용 시스템을 통합한 VR 화재 대피 및 소화기 훈련 시스템을 개발하였다.

개발된 시스템은 다음과 같은 기여를 할 것으로 기대 된다.

- 학습자의 훈련 절차 수행을 실시간으로 평가하고 오류를 감지하여 즉각적인 피드백을 제공함으로써, 반복 학습을 통한 숙련도 향상에 기여함.
- AI Agent를 통해 사용자가 훈련 도중 자유롭게 질문하고, 필요한 정보를 음성과 시각으로 제공받을 수 있는 구조를 갖추으로써, 훈련 중단 없이 자율 학습이 가능한 환경을 제공함.
- 훈련 결과 요약 UI(End UI)를 통해 사용자의 절차 수행 이력을 시각적으로 확인할 수 있도록 하여, 피드백 기반 자기 점검과 반복 학습의 기반을 마련함.

이러한 구조는 단순한 시뮬레이션을 넘어, 사용자의 학습 과정 자체를 분석하고 반응하는 인터랙티브 교육 시스템의 방향성을 제시한다는 점에서 의미가 깊다.

5.2. 향후 연구 방향

향후에는 다음과 같은 방향으로 본 연구 시스템을 확장하고 개선할 수 있다.

- **사용자 행동 분석 기반 평가 기능 강화**
현재는 절차 중심 평가에 집중되어 있으나, 사용자의 반응 시간, 머리 및 손 위치, 경로 최적성 등 행동 데이터를 기반으로 한 정량적 평가 시스템으로 발전시킬 수 있다.
- **훈련 콘텐츠 다양화**
화재 대피 외에도 지진, 산업 안전, 교통사고 대처 등 다양한 재난 상황에 적용 가능한 시나리오를 구축함으로써 VR 기반 종합 재난 대응 교육 플랫폼으로 확장할 수 있다.
- **훈련 리포트 자동 생성 기능**
훈련 종료 후 사용자 행동 이력을 기반으로 PDF 리포트 자동 생성 기능을 도입하여, 사용자나 기관이 학습 결과를 문서화하고 관리할 수 있도록 지원할 수 있다.
- **협동형 멀티 유저 훈련**
단일 사용자 훈련을 넘어서, 여러 사용자가 동시에 협업하는 훈련 시나리오를 개발함으로써, 집단 대피, 역할 분담 상황 등의 복합 시나리오 학습이 가능해질 것이다.

6. 구성원별 역할 및 개발 일정

6.1. 개발 일정

작업		3월				4월				5월				6월		
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3
1	Task 추상화 및 설계															
2	Task 구현															
3	시연 주제 결정															
4	중간 보고서 작성															
5	교육 시스템 설계															
6	모듈 테스트 및 개선															
7	모듈 조립 및 시스템 생성															
8	시스템 디버깅 및 개선															
9	시스템 배포															
10	최종보고서 작성															
11	발표자료 준비															

Table 8 개발 일정

6.2. 역할 분담

이름	담당 역할	주요 업무
강원우	썬 제작, UI/UX 제작, 프로젝트 매니저, AI Agent 개발	프로젝트 일정 관리, 대피 썬 구성 및 디자인, 사용자 인터페이스 설계, AI 에이전트 개발, 최종 보고서 작성
조현진	썬 제작, 에셋 제작, 메모리 최적화, 데이터 수집 및 분석	전체 프로젝트 관리, 소화기 썬 구성 및 디자인, 3D 모델링, 메모리 최적화 작업, 데이터 수집

Table 9 역할 분담

7. 참고 문헌

- [1] 최지원, 김영상, 송동석, 용환성. (2024). 방사선비상진료 대응 VR교육을 위한 VR교육 모니터링시스템에관한연구.디지털콘텐츠학회논문지,25(7),1861-1871.
- [2] 정경환, 박정규. (2024). 방사선학과 학생들의 VR 교육만족도조사.한국콘텐츠학회논문지, 24(7), 473-480. [9] 최섭. (2021). VR 기반 프로그램을 활용한 과학적 모형 구성 수업의 개발과 효과(박사학위논문). 서울대학교대학원.
- [3] Shin, M. & Cho, Y. (2019). Comparison of Traditional vs. VR Fire Drill Training. Fire Engineering Review, 13(1), 52–60.
- [4] Park, S. et al. (2020). Development of Radiation Emergency Simulation Training Using Virtual Reality. Journal of Disaster & Safety, 15(2), 102–110.
- [5] Choi, H., Lim, Y., & Jeong, S. (2021). Real-Time Procedural Feedback System for Surgical VR Training: Effects on Error Recognition and Skill Retention. Journal of Simulation in Healthcare, Vol. 10, No. 4, pp. 212–220.
- [6] Lee, J., & Choi, H. (2020). Effects of simulation-based education for neonatal resuscitation on medical students' technical and non-technical skills. PLoS One, 17(12), e0278575. <https://doi.org/10.1371/journal.pone.0278575>
- [7] Kwon, D., Han, J., & Lee, M. (2019). Process Training in Virtual Reality: A Study on User Confusion and Feedback Design. Virtual Training & Simulation Review, 6(2), 87–95.
- [8] 국가법령정보센터. 『화재 및 피난 시뮬레이션의 시나리오 작성 기준』, 소방청 고시 제 2021-1호, 2021.
- [9] 경기도소방재난본부. 『원테이크 119초 안전교육 – 화재 대피편』, YouTube 영상. (<https://youtu.be/KgCaXeNK8IU?feature=shared>)
- [10] Kim, H., Park, J., & Lee, S. (2022). Effectiveness of VR-based Emergency Response Training: A Comparative Study. Journal of Educational Technology, 38(4), 215–229.
- [11] Google Developers. (2024). Text-to-Speech API. <https://cloud.google.com/text-to-speech>
- [12] Google Developers. (2024). Gemini Pro API. <https://ai.google.dev/gemini>
- [13] Oculus Developer Documentation. (2023). Designing for VR Training. <https://developer.oculus.com>

[14] 김용구, 김태언. (2023). 교육참여자 모듈별 문제해결방식 적용을 통한 VR 방식 안전 교육개선예관한연구.한국산학기술학회추계학술발표논문집.

[15] kookmin-sw. (n.d.). SAFE LAB: VR 실험실 안전교육.GitHub.RetrievedFebruary 20, 2025, <https://github.com/kookmin-sw/capstone-2020-17>